Informe Proyecto de Inteligencia Artificial (HEX)

Facultad de Matemática y Computación (MATCOM)

Autor:
Fabio Víctor Alonso Bañobre

Curso: 2025 - 2026

Introducción

HEX es un juego de estrategia de tablero para dos jugadores, inventado de manera independiente por Piet Hein y John Nash. Se juega sobre una cuadrícula de celdas hexagonales, típicamente de tamaño 11×11, pero en el proyecto se considera cualquier tablero de $N \times N$.

A diferencia de otros juegos de mesa, el HEX nunca puede terminar en empate. El tablero es un rombo compuesto por hexágonos donde cada jugador tiene asignados dos lados opuestos del perímetro. Las reglas son sencillas pero permiten una profundidad estratégica compleja:

- Los jugadores alternan turnos para colocar una ficha de su color en cualquier casilla que esté vacía.
- El primer jugador (Jugador Nº 1) debe conectar los lados izquierdo y derecho del tablero mediante una cadena ininterrumpida de sus piezas.
- El segundo jugador (Jugador Nº 2) debe conectar los lados superior e inferior de manera vertical.
- Las conexiones son válidas en cualquiera de las seis direcciones adyacentes de cada hexágono.

Objetivo del Proyecto

El objetivo principal de este proyecto es el diseño, implementación y optimización de un agente inteligente capaz de jugar de manera autónoma y competitiva al HEX. El desarrollo se centra en los siguientes puntos claves:

- **Toma de Decisiones bajo Restricciones:** Implementar algoritmos de búsqueda en espacios de estados que permitan seleccionar la jugada óptima o subóptima dentro de un límite de tiempo estricto de 5 segundos por movimiento.
- **Eficiencia Algorítmica:** Desarrollar estructuras de datos y funciones de evaluación heurística que minimicen la complejidad computacional, evitando crecimientos exponenciales ineficientes en tableros de grandes dimensiones.
- **Hibridación de Estrategias:** Crear un sistema capaz de adaptar su lógica de juego según el contexto, utilizando técnicas de búsqueda exhaustiva (como Minimax) para precisión en tableros pequeños y métodos probabilísticos (como Monte Carlo Tree Search) para manejar la explosión combinatoria en tableros extensos.
- **Cumplimiento de Requerimientos Técnicos:** Asegurar que el agente herede correctamente de la clase base proporcionada, respetando la idempotencia y el aislamiento del sistema para su correcto funcionamiento en un entorno de torneo automatizado.

Estrategia Híbrida de Búsqueda

La elección de no depender de un único algoritmo nace de la naturaleza matemática del juego HEX. A diferencia del ajedrez, donde el factor de ramificación es cercano a 35, en HEX este factor es del orden N^2 en el inicio de la partida, decreciendo conforme se ocupan casillas. Esto significa que en un tablero de 20×20 tenemos hasta 400 movimientos posibles en el primer turno. Esta diferencia radical de escenarios según el tamaño del tablero (N) nos llevó a concluir que una estrategia única sería ineficiente, estableciendo el umbral en $N \leq 11$ para búsqueda exacta y $N > 11$ para búsqueda estocástica.

Criterio	Minimax + $\alpha\beta$	MCTS
Régimen	$N \leq 11$	$N > 11$
Celdas máximas	121	> 144
Tipo de búsqueda	Exhaustiva determinista	Estocástica guiada
Profundidad	Acotada por tiempo	Ilimitada (por simulación)
Fortaleza	*Endgame* exacto	Apertura y medio juego
Debilidad	Explosión combinatoria	Requiere muchas simulaciones

Table 1: Comparativa de regímenes de búsqueda según tamaño del tablero.

Escenario de Precisión: Minimax con Poda Alpha-Beta ($N \leq 11$

En tableros pequeños y medianos, el espacio de búsqueda, aunque grande, es lo suficientemente manejable para que un algoritmo determinista brinde una ventaja decisiva. Se implementó **Iterative Deepening Alpha-Beta (IDAB)**: en lugar de fijar una profundidad máxima, el algoritmo itera desde profundidad 1 hasta la máxima alcanzable dentro del presupuesto de tiempo, garantizando que siempre exista una jugada válida disponible incluso si el tiempo expira.

Ventajas clave:

- **Determinismo táctico:** En estas dimensiones, **Minimax** puede explorar lo suficientemente profundo como para identificar conexiones virtuales y puentes de manera exacta, sin depender de simulaciones probabilísticas.
- **Cierre de partida:** **Minimax** es excepcionalmente efectivo en el *endgame*. Si existe una secuencia de jugadas que garantiza la victoria, la encontrará y ejecutará sin errores.
- **Optimización por poda $\alpha\beta$:** La Poda **Alpha-Beta** permite ignorar ramas enteras del árbol cuando se detecta que no pueden mejorar la solución ya conocida, maximizando el aprovechamiento del límite de tiempo de 5 segundos. En el mejor caso, reduce la complejidad de $O(b^d)$ a $O(b^{d/2})$, equivalente a duplicar la profundidad alcanzable.
- **Memoria eficiente:** A diferencia de algoritmos como BFS completo, el árbol **Minimax** solo mantiene en memoria el camino activo, lo que lo hace viable para tableros de hasta 11×11.

Escenario de Explosión Combinatoria: MCTS ($N > 11$)

Cuando el tablero supera $N = 11$, Minimax sufre de "miopía combinatoria": el árbol de búsqueda crece de forma exponencial y el algoritmo solo puede profundizar unos pocos niveles, perdiéndose conexiones estratégicas a largo plazo. En este escenario, el **Monte Carlo Tree Search (MCTS)** resulta superior por su capacidad de construir un árbol asimétrico guiado por resultados estadísticos.

El criterio de selección dentro del árbol MCTS se rige por la fórmula **UCB1 (Upper Confidence Bound)**, que balancea la explotación de ramas prometedoras con la exploración de ramas poco visitadas:

$$\text{UCB1}(v) = \frac{w_v}{n_v} + C \cdot \sqrt{\frac{\ln N_v}{n_v}} \tag{1}$$

donde w_v es el número de victorias acumuladas en el nodo v, n_v es el número de veces que fue visitado, N_v es el número de visitas del nodo padre, y C es la constante de exploración. En nuestra implementación se usa $C = 0.8 < \sqrt{2}$, favoreciendo la explotación dado que nuestro rollout no es aleatorio sino informado por BFS, lo que hace que las estimaciones sean más fiables y requieran menos exploración adicional.

Ventajas clave:

- **Búsqueda asimétrica:** A diferencia de Minimax, que expande el árbol de forma uniforme, **MCTS** concentra las simulaciones en las ramas estadísticamente más prometedoras, alcanzando profundidades que Minimax no podría explorar en el tiempo disponible.

- **Rollout informado por BFS:** En lugar de simular partidas completamente aleatorias, nuestro rollout estima el resultado mediante distancias de camino mínimo (0-1 BFS), produciendo estimaciones de calidad en $O(N^2)$ por simulación y permitiendo miles de iteraciones dentro del límite de tiempo.
- **Escalabilidad temporal: MCTS** aprovecha cada milisegundo disponible. Cuanto más tiempo tiene, más simulaciones ejecuta y más precisas son sus estimaciones estadísticas, sin colapsar por limitaciones de memoria.
- **Adaptabilidad sin función de evaluación perfecta:** En tableros de 19×19, diseñar una heurística matemáticamente óptima es extremadamente difícil. **MCTS** reduce esta dependencia mediante el muestreo estadístico de posiciones.

Función de Evaluación y Heurísticas

La función de evaluación es el mecanismo mediante el cual el agente cuantifica la calidad de una posición sin necesidad de explorar el árbol hasta un estado terminal. Su diseño se basa en los principios de **Búsqueda Informada** del Bloque 1: una heurística admisible $h(n)$ que nunca sobreestime el coste real de alcanzar el objetivo.

La evaluación final combina cinco componentes ponderados:

$$\text{eval}(s) = \big(w_1 \cdot \Delta d + w_2 \cdot \Delta\text{VC} + w_3 \cdot \Delta\text{CC} + w_4 \cdot \Delta\text{BP} + w_5 \cdot \Delta\text{Centro}\big) \cdot (0.5 + \alpha) \tag{2}$$

- s: estado actual del tablero evaluado.
- Δd: diferencia de distancias mínimas de conexión (oponente − propio).
- ΔVC: diferencia de conexiones virtuales (propio − oponente).
- ΔCC: diferencia del tamaño de la mayor componente conexa (propio − oponente).
- ΔBP: diferencia del potencial de puente (propio − oponente).
- ΔCentro: diferencia del control del centro (propio − oponente).
- $w_1 = 1.0,\ w_2 = 0.35,\ w_3 = 0.20,\ w_4 = 0.15,\ w_5 = 0.08$: pesos de cada componente.
- α: parámetro de agresividad adaptativa según rol del jugador.

Componente 1: Distancia de Camino Mínimo (0-1 BFS)

El componente principal de la evaluación es la diferencia de distancias mínimas de conexión entre ambos jugadores, calculada mediante **0-1 BFS**, una variante del algoritmo UCS (*Uniform Cost Search*) con costes binarios:

- **Coste 0** → celda propia: atravesarla es gratuito, ya forma parte del camino de conexión.
- **Coste 1** → celda vacía: hay que colocar una pieza para usarla.
- **Bloqueada** → celda del oponente: no puede formar parte del camino propio.

Esta heurística es **admisible**: la distancia calculada corresponde al número mínimo de piezas adicionales necesarias para completar la conexión, por lo que nunca sobreestima el esfuerzo real. La implementación usa una *deque* de doble extremo: los nodos de coste 0 se insertan al frente (*appendleft*) y los de coste 1 al fondo (*append*), garantizando que siempre se procesa primero el nodo de menor coste acumulado en $O(N^2)$.

El *score* de distancia se define como:

$$\Delta d = d_{\text{oponente}} - d_{\text{propio}} \tag{3}$$

- d_{propio}: distancia mínima del jugador actual para conectar sus dos bordes, calculada por **0-1 BFS** desde el borde fuente hasta el borde objetivo.
- d_{oponente}: distancia mínima del oponente para conectar sus dos bordes.

Un valor positivo indica ventaja propia (el oponente necesita más pasos para conectar), y uno negativo indica desventaja. Este componente es el más significativo de la evaluación con peso $w_1 = 1.0$.

Componente 2: Conexiones Virtuales

Una **conexión virtual** es un par de piezas propias que comparten exactamente dos celdas vacías como vecinas en común. Esta estructura garantiza la conexión entre ambas piezas independientemente de la jugada del oponente: si el oponente ocupa una de las dos celdas mediadoras, el jugador puede responder ocupando la otra, manteniendo la conexión. Matemáticamente, para un par de celdas propias (c_1, c_2):

$$\text{VC}(c_1, c_2) = 1 \iff |\mathcal{N}(c_1) \cap \mathcal{N}(c_2) \cap \text{Vacías}| \geq 2 \tag{4}$$

- c_1, c_2: par de celdas ocupadas por el jugador propio.
- $\mathcal{N}(c)$: conjunto de hasta 6 vecinos hexagonales de la celda c en el layout *even-r*.
- $\mathcal{N}(c_1) \cap \mathcal{N}(c_2) \cap \text{Vacías}$: celdas vacías compartidas como vecinas por ambas piezas (celdas mediadoras del puente virtual).

La evaluación premia tener más conexiones virtuales propias y penaliza las del oponente con peso diferenciado ($w_2 = 0.35$ para las propias, $w_{\text{opp}} = 0.30$ para las del oponente), reconociendo que bloquear conexiones del rival es ligeramente menos prioritario que construir las propias.

Componente 3: Componente Conexa Mayor

Este componente mide el tamaño de la **mayor componente conexa** de piezas propias mediante DFS. Una componente conexa grande indica que el jugador tiene un grupo consolidado de piezas que ya están conectadas entre sí, lo que facilita extender la conexión hacia los bordes objetivo. La diferencia entre la componente mayor propia y la del oponente ($w_3 = 0.20$) captura quién tiene la estructura más desarrollada en el tablero.

Componente 4: Potencial de Puente

El **potencial de puente** es una señal estructural que cuantifica cuántos pares de piezas propias se encuentran a distancia 1 o 2 en el grafo hexagonal:

- Par a **distancia 1** (vecinos directos): contribuye con +2.0 al *score*.
- Par a **distancia 2** (vecinos de vecinos): contribuye con +1.0 al *score*.

Este componente premia posiciones con alta conectividad potencial, donde las piezas están bien distribuidas y próximas entre sí, aunque aún no formen una conexión directa. Con peso $w_4 = 0.15$, actúa como señal de "densidad táctica" de la posición.

Componente 5: Control del Centro

Las celdas centrales del tablero hexagonal tienen mayor conectividad estructural: desde el centro se puede alcanzar cualquier borde con menos pasos que desde las esquinas. El control del centro se evalúa como:

$$\text{Centro}(p) = w_5 \cdot \sum_{(r,c) \in \text{piezas}(p)} \left(N - |r - \tfrac{N}{2}| - |c - \tfrac{N}{2}|\right) \tag{5}$$

- p: jugador cuyas piezas se están evaluando.
- r, c: fila y columna de cada pieza colocada por el jugador.

- N: tamaño del tablero.
- $N - |r - \frac{N}{2}| - |c - \frac{N}{2}|$: proximidad al centro de la celda (r, c); toma valor máximo N en el centro y decrece linealmente hacia los bordes.
- $w_5 = 0.08$: peso del componente, diseñado como factor de desempate suave.

Agresividad Adaptativa

El factor $(0.5 + \alpha)$ escala el resultado completo de la evaluación. El parámetro α varía según el rol del jugador:

- **Jugador 1** ($\alpha = 0.72$): perfil más agresivo, escala la evaluación por 1.22. Como primer jugador tiene ventaja estructural en Hex y puede permitirse priorizar el avance propio sobre el bloqueo.
- **Jugador 2** ($\alpha = 0.58$): perfil más conservador, escala por 1.08. El segundo jugador necesita equilibrar más el avance propio con el bloqueo del oponente.

Ordenamiento Greedy de Movimientos

Complementaria a la función de evaluación, la **búsqueda Greedy** del Bloque 1 ordena los movimientos candidatos antes de explorarlos, maximizando la probabilidad de que las primeras ramas exploradas sean las más prometedoras. El *score greedy* de cada celda vacía (r, c) se calcula como:

$$\text{score}(r, c) = 3 \cdot n_{\text{propios}} + 2 \cdot n_{\text{oponente}} + \beta \cdot \left(N - |r - \tfrac{N}{2}| - |c - \tfrac{N}{2}|\right) + \gamma \cdot H(r, c) \tag{6}$$

- n_{propios}: número de vecinos hexagonales de (r, c) ocupados por piezas propias.
- n_{oponente}: número de vecinos hexagonales de (r, c) ocupados por piezas del oponente.
- β: sesgo de centralidad (1.45 para Jugador 1, 1.20 para Jugador 2).
- $H(r, c)$: *score* acumulado de la *History Heuristic* para la celda (r, c), incrementado en depth^2 cada vez que una jugada en esa celda causó un corte en la búsqueda.
- $\gamma = 0.0007$: factor de escala de la *History Heuristic*, evita que domine sobre los componentes tácticos locales.

Este ordenamiento es fundamental para la eficiencia de la Poda Alpha-Beta: si los mejores movimientos se exploran primero, el número de cortes se maximiza, reduciendo el árbol efectivo de $O(b^d)$ hacia $O(b^{d/2})$.

Adicionalmente, para el régimen MCTS en tableros grandes ($N > 11$), el ordenamiento de candidatos aplica el concepto de **frontera BFS** (Bloque 1): en lugar de escanear las N^2 celdas del tablero, solo se consideran las celdas vacías *adyacentes* a piezas ya colocadas. Este enfoque, análogo a cómo BFS expande únicamente desde la frontera actual sin revisar nodos ya procesados, reduce el coste de generación de candidatos de $O(N^2)$ a $O(k \cdot 6)$ donde k es el número de piezas colocadas, aumentando significativamente el número de simulaciones MCTS posibles dentro del límite de tiempo.

Búsqueda Adversarial y Optimizaciones

La búsqueda adversarial modela el juego HEX como un problema de decisión entre dos agentes racionales con objetivos opuestos. Bajo el paradigma de **juego suma-cero** del Bloque 2, cada ganancia del jugador propio implica una pérdida equivalente para el oponente, lo que permite modelar ambas perspectivas con una única función de utilidad: el jugador propio la *maximiza* y el oponente la *minimiza*.

Algoritmo Minimax con Poda Alpha-Beta

El algoritmo **Minimax** determina la jugada óptima asumiendo que ambos jugadores actúan de forma perfectamente racional. La recursión alterna entre nodos maximizadores (turno propio) y minimizadores (turno del oponente):

$$\text{Minimax}(s, d) = \begin{cases} \text{eval}(s) & \text{si } d = 0 \text{ o } s \text{ es terminal} \\ \max\limits_{a \in A(s)} \text{Minimax}(s_a, d-1) & \text{si maximizando} \\ \min\limits_{a \in A(s)} \text{Minimax}(s_a, d-1) & \text{si minimizando} \end{cases} \tag{7}$$

- s: estado actual del tablero.
- d: profundidad restante de búsqueda.
- $A(s)$: conjunto de acciones disponibles en el estado s.
- s_a: estado resultante de aplicar la acción a en s.
- $\text{eval}(s)$: función de evaluación heurística descrita en el punto anterior.

La **Poda Alpha-Beta** optimiza Minimax eliminando ramas del árbol que no pueden influir en la decisión final, sin alterar el resultado. Mantiene dos cotas durante la búsqueda:

$$\alpha = \max \text{ garantizado por el maximizador} \qquad \beta = \min \text{ garantizado por el minimizador} \tag{8}$$

- **Corte** β: si en un nodo maximizador se encuentra val $\geq \beta$, el minimizador nunca elegiría este camino. Se abandona la exploración de los hermanos restantes.
- **Corte** α: si en un nodo minimizador se encuentra val $\leq \alpha$, el maximizador nunca elegiría este camino. Se abandona la exploración de los hermanos restantes.

En el mejor caso, la poda reduce la complejidad de $O(b^d)$ a $O(b^{d/2})$, equivalente a duplicar la profundidad explorable en el mismo tiempo. Este mejor caso se alcanza cuando los movimientos están perfectamente ordenados, lo que motiva directamente las técnicas de ordenamiento descritas a continuación.

Iterative Deepening Alpha-Beta (IDAB)

En lugar de fijar una profundidad máxima estática, el agente implementa **Iterative Deepening Alpha-Beta**: itera profundidades $d = 1, 2, \ldots, d_{\max}$ dentro del presupuesto de tiempo disponible. Tras cada iteración completa, la mejor jugada encontrada se conserva como respaldo. Si el tiempo expira durante una iteración, se devuelve la mejor jugada de la iteración anterior completada.

La profundidad máxima $d_{\max}$ y el tamaño del *beam* de candidatos raíz se adaptan al tamaño del tablero y a la fase de juego:

N	$d_{\max}$ **(apert.)**	$d_{\max}$ **(resto)**	**Beam (apert.)**	**Beam (resto)**
≤ 5	7	9	18	18
≤ 7	6	8	16	16
≤ 9	5	7	14	16
≤ 11	4	6	12	14

Table 2: Parámetros adaptativos de búsqueda según tamaño del tablero y fase de juego.

Adicionalmente, tras cada iteración completada, la mejor jugada encontrada se mueve al frente de la lista de candidatos raíz para la siguiente iteración, mejorando el ordenamiento progresivamente y maximizando los cortes alpha-beta en iteraciones posteriores.

Tabla de Transposición con Hashing Zobrist

El árbol Minimax contiene numerosas **posiciones repetidas**: secuencias de jugadas distintas que llevan al mismo estado del tablero. Sin memoización, estas posiciones se recomputan innumerables veces. La **Tabla de Transposición** almacena los resultados ya evaluados indexados por un hash de la posición.

El **Hashing Zobrist** asigna a cada combinación (jugador, fila, columna) un número aleatorio de 64 bits generado con semilla fija. El *hash* de cualquier posición es el XOR de los valores correspondientes a todas las piezas colocadas:

$$h(s) = \bigoplus_{(p,r,c)\in s} z_{p,r,c} \tag{9}$$

- $h(s)$: *hash* de 64 bits del estado s.
- $z_{p,r,c}$: número aleatorio de 64 bits asignado al jugador p en la celda (r, c).
- $\bigoplus$: operación XOR sobre todos los valores de las piezas presentes.

La propiedad clave del XOR es que actualizar el *hash* tras una jugada es $O(1)$: basta aplicar $h' = h \oplus z_{p,r,c}$, evitando recomputar el *hash* completo del tablero en cada nodo del árbol. En nuestra implementación, $h(s)$ se calcula completamente en $O(N^2)$ una única vez al inicio de cada turno; todas las actualizaciones posteriores dentro del árbol son $O(1)$.

La tabla de transposición almacena cuatro campos por entrada: el valor evaluado, la profundidad a la que fue calculado, el **tipo de cota** y la mejor jugada encontrada:

Tipo de cota	Condición	Uso
`EXACT`	Ventana $[\alpha, \beta]$ no saturada	Devolver directamente
`LOWER_BOUND`	Corte β: valor real $\geq$ almacenado	Actualizar α
`UPPER_BOUND`	All-node: valor real $\leq$ almacenado	Actualizar β

Table 3: Tipos de cota almacenados en la tabla de transposición.

El uso de cotas permite al algoritmo estrechar la ventana $[\alpha, \beta]$ incluso cuando no puede devolver el valor exacto, generando más cortes que una tabla que solo almacene valores exactos. Adicionalmente, la mejor jugada almacenada (*TT-move*) se prueba siempre en primer lugar cuando se vuelve a visitar el nodo, concentrando los cortes en los movimientos históricamente mejores.

Killer Moves

La **heurística de Killer Moves** explota una observación empírica de la búsqueda adversarial: una jugada que causó un corte β en un nodo hermano probablemente cause otro corte en el nodo actual del mismo nivel de profundidad, ya que ambos comparten el mismo contexto estratégico.

Para cada nivel de profundidad d desde la raíz, se mantiene una lista FIFO de hasta 2 *killer moves*. El *pipeline* de ordenamiento en cada nodo interno es:

1. **TT-move**: jugada almacenada en la tabla de transposición para este hash*hash* (si existe).
2. **Killer moves**: jugadas que causaron cortes en nodos hermanos del mismo nivel (si la celda sigue vacía).
3. **Resto**: candidatos ordenados por *score greedy* + *History Heuristic*.

Cada vez que una jugada provoca un corte alpha o beta, se registra mediante `_register_cutoff`, que actualiza simultáneamente los *killers* y la History Heuristic del siguiente apartado.

History Heuristic

La **History Heuristic** mantiene un diccionario global $H : (r, c) \to R$ que acumula evidencia estadística sobre qué celdas tienden a causar cortes a lo largo de la búsqueda. Cada vez que una jugada en la celda (r, c) provoca un corte a profundidad d restante, su *score* se incrementa:

$$H(r, c) \mathrel{+}= d^2 \tag{10}$$

- $H(r, c)$: *score* acumulado de la celda (r, c) en la History Heuristic.
- d: profundidad restante en el momento del corte. El factor d^2 pondera más los cortes en nodos profundos, donde la evidencia es más valiosa.

A diferencia de los Killer Moves, que son locales a un nivel de profundidad y se reinician en cada iteración del IDAB, la History Heuristic persiste durante toda la partida, acumulando conocimiento sobre qué posiciones del tablero son tácticamente activas. Su contribución al ordenamiento *greedy* está escalada por $\gamma = 0.0007$ para evitar que domine sobre los componentes tácticos locales.

La combinación de TT-move, Killer Moves e History Heuristic en el *pipeline* de ordenamiento produce un efecto compuesto: en posiciones bien estudiadas, los primeros movimientos explorados en cada nodo son consistentemente los mejores, maximizando la tasa de cortes y permitiendo que IDAB alcance profundidades significativamente mayores dentro del límite de tiempo de 5 segundos.

Razonamiento CSP y Planificación STRIPS

Más allá de la búsqueda adversarial, el agente incorpora dos marcos de razonamiento adicionales del curso para enriquecer la calidad de sus decisiones: los **Problemas de Satisfacción de Restricciones** (Bloque 3) para detectar jugadas forzadas, y la **Planificación Automática STRIPS** (Bloque 5) para ordenar las acciones disponibles según su relevancia al objetivo.

Modelado del Turno como CSP

Cada turno del juego se modela como un CSP con la siguiente estructura:

- **Variables**: celdas vacías del tablero $X = \{x_1, x_2, \ldots, x_k\}$.
- **Dominios**: cada variable puede tomar el valor $\{\text{jugador_propio}, \text{oponente}\}$.
- **Restricción**: la asignación debe mantener $h(n) \leq d^*$, donde d^* es la distancia óptima actual calculada por 0-1 BFS.

Bajo este modelo, una celda vacía es **relevante** si y solo si su asignación puede reducir o mantener $h(n)$. Las celdas que no pertenecen a ningún camino óptimo no satisfacen la restricción y pueden ignorarse sin pérdida de optimalidad.

BFS Bidireccional: Path Cells y Landmarks

Para identificar las celdas relevantes del CSP, el agente ejecuta un **BFS bidireccional**: un BFS desde la fuente (`dist_f`) y otro desde el objetivo (`dist_b`). Una celda (r, c) pertenece a algún camino óptimo si y solo si:

$$\texttt{dist_f}[r][c] + \texttt{dist_b}[r][c] - \text{cost}(r, c) = d^* \tag{11}$$

- $\texttt{dist_f}[r][c]$: coste mínimo desde el borde fuente hasta la celda (r, c).
- $\texttt{dist_b}[r][c]$: coste mínimo desde la celda (r, c) hasta el borde objetivo.
- $\text{cost}(r, c)$: coste de la celda (0 si es propia, 1 si está vacía); se descuenta porque aparece contado en ambos BFS.
- d^*: distancia óptima total del jugador evaluado.

El conjunto de celdas que satisfacen esta condición se denomina **path cells**: son las acciones útiles del plan actual. Este enfoque bidireccional evita ejecutar un BFS independiente desde cada celda, reduciendo el coste total de $O(N^4)$ a $O(N^2)$ mediante dos BFS completos.

Entre las *path cells* vacías, una celda es **landmark** si bloquearla incrementa d^*:

$$(r, c) \in \text{Landmarks} \iff d^*\big(s \cup \{(r, c) \to \text{oponente}\}\big) > d^* \tag{12}$$

- $s \cup \{(r, c) \to \text{oponente}\}$: estado del tablero con la celda (r, c) temporalmente bloqueada por el oponente.

Un *landmark* es una celda que aparece en *toda* asignación óptima del CSP: si se bloquea, ninguna combinación de las variables restantes puede satisfacer la restricción $h(n) \leq d^*$. Esto corresponde directamente al concepto de **arc-consistency** del Bloque 3: bloquear un *landmark* reduce el dominio factible del oponente a vacío, haciendo la restricción insatisfacible.

Detección de Jugadas Forzadas (Arc-Consistency)

El agente aplica razonamiento CSP antes de invocar Minimax o MCTS para detectar jugadas que deben ejecutarse o bloquearse inmediatamente, sin necesidad de búsqueda completa:

1. **Victoria inmediata**: si existen $N-2$ o más piezas propias en el tablero, se verifica si alguna celda vacía completa la conexión. Coste: $O(k)$ clones donde k es el número de celdas vacías candidatas.

2. **Bloqueo inmediato**: análogamente, si el oponente tiene $N-2$ o más piezas, se detecta si puede ganar en el siguiente turno y se bloquea. El umbral $N-2$ en lugar de $N-1$ amplía la detección a victorias casi inmediatas, aumentando la robustez táctica.

3. **Landmark singleton** (solo en $N \leq 11$): si el oponente tiene exactamente 1 *landmark* vacío en fase media (opp_count $\geq N/2$), bloquear esa celda es obligatorio por *arc-consistency*. El dominio factible del oponente tiene cardinalidad 1: existe una única celda cuya ocupación mantiene su restricción satisfacible. Bloquearla la viola, forzando $d^* \to \infty$ para el oponente.

La detección de *landmark singleton* está restringida a $N \leq 11$ porque en tableros grandes el coste de calcular *landmarks* ($O(N^2)$ por celda de path) destruiría el *throughput* de simulaciones MCTS, donde el volumen de iteraciones es el factor determinante del rendimiento.

Modelado del Turno como Problema STRIPS

Cada turno se modela también como un problema de **Planificación STRIPS** del Bloque 5:

- **Estado inicial**: configuración actual del tablero.
- **Objetivo**: conectar los dos bordes propios.
- **Acciones** `place_piece`(r, c):
  - *Precondición*: celda (r, c) está vacía.
  - *Efecto Add*: celda (r, c) pasa a ser propia.

Las **path cells** son las acciones necesarias del plan **STRIPS** actual: son las únicas acciones que reducen $h(n)$ hacia el objetivo. Las **landmarks** son las acciones *necesarias* en todo plan ganador: cualquier secuencia de acciones que alcance el objetivo debe incluirlas. Este concepto proviene directamente de la teoría de *landmarks* en planificación automática.

Ordering de Acciones por Relevancia al Plan

La combinación del razonamiento CSP y STRIPS produce un **ordenamiento jerárquico de acciones** para la raíz de Minimax ($N \leq 11$) que prioriza las jugadas según su relevancia al plan ganador actual:

Tier	Tipo de celda	Justificación
1	Mi landmark (+10 000)	Acción necesaria en todo plan ganador propio.
2	Landmark del oponente (+9 000)	Bloquear la acción necesaria del plan enemigo.
3	Mi path cell (+5 000)	Acción que avanza el plan propio actual.
4	Path cell del oponente (+4 000)	Acción que interfiere el plan enemigo.
5	Resto (+0)	Ordenado por score greedy + History Heuristic.

Table 4: Jerarquía de ordenamiento de acciones basada en landmarks y path cells.

El bonus de *tier* se suma al *score greedy* base, garantizando que las acciones de mayor relevancia estratégica se exploren siempre primero en el árbol Minimax. Esto maximiza la tasa de cortes alpha-beta desde la raíz, donde cada poda ahorra más trabajo que en cualquier otro nivel del árbol.

La integración de los tres marcos de razonamiento — búsqueda informada (Bloque 1), CSP (Bloque 3) y planificación STRIPS (Bloque 5) — en el *pipeline* de decisión del agente produce un efecto sinérgico: las mismas estructuras BFS que calculan la heurística de evaluación sirven simultáneamente para identificar *landmarks* del plan y ordenar las acciones por relevancia, sin coste computacional adicional significativo.

Metaheurística MCTS para Tableros Grandes

Para tableros con $N > 11$, la explosión combinatoria hace inviable una búsqueda exhaustiva. El agente adopta **Monte Carlo Tree Search (MCTS)** como metaheurística de trayectoria del Bloque 4: en lugar de explorar el árbol de forma determinista, construye progresivamente un árbol asimétrico guiado por resultados estadísticos de simulaciones, con criterio de parada por tiempo.

MCTS como Metaheurística de Trayectoria

En el marco del Bloque 4, MCTS es una **metaheurística de trayectoria**: parte de un estado inicial (raíz del árbol) y construye una solución paso a paso, donde cada paso es una simulación completa desde la raíz hasta una estimación terminal. A diferencia de las metaheurísticas poblacionales como los Algoritmos Genéticos, MCTS mantiene un único árbol que se refina iterativamente.

El equilibrio fundamental del Bloque 4 entre **exploración** y **explotación** se materializa directamente en MCTS:

- **Explotación**: profundizar en ramas que han demostrado estadísticamente ser buenas (alto winrate).
- **Exploración**: visitar ramas poco exploradas que podrían ser mejores de lo que sugieren sus estadísticas actuales.

Este balance se regula mediante la constante C de la fórmula UCB1. En nuestra implementación se usa $C = 0.8 < \sqrt{2}$, favoreciendo la explotación dado que el *rollout* es informado por BFS y produce estimaciones fiables, reduciendo la necesidad de exploración adicional.

Las Cuatro Fases del MCTS

Cada iteración del algoritmo ejecuta cuatro fases sobre el árbol construido hasta ese momento:

Fase 1: Selección

Desde la raíz, se desciende por el árbol eligiendo en cada nodo el hijo con mayor valor UCB1 hasta alcanzar un nodo no completamente expandido o un estado terminal:

$$\text{UCB1}(v) = \frac{w_v}{n_v} + C \cdot \sqrt{\frac{\ln N_v}{n_v}} \tag{13}$$

- w_v: victorias acumuladas en el nodo v.
- n_v: número de visitas al nodo v.
- N_v: número de visitas al nodo padre de v.
- $C = 0.8$: constante de exploración.

Si $n_v = 0$, UCB1 devuelve $+\infty$, garantizando que todo nodo sea visitado al menos una vez antes de que UCB1 comience a discriminar entre ellos.

Fase 2: Expansión

Al alcanzar un nodo con movimientos no explorados (`untried_moves`), se extrae el primer candidato de la lista ordenada, se crea un nuevo nodo hijo y se añade al árbol. Los `untried_moves` de la raíz se inicializan con el ordenamiento *greedy* del Bloque 1, priorizando las celdas más prometedoras. Los nodos internos usan la **frontera BFS**: solo las celdas vacías adyacentes a piezas ya colocadas, reduciendo el espacio de expansión de $O(N^2)$ a $O(k \cdot 6)$ sin perder los candidatos relevantes.

Fase 3: Simulación (Rollout)

Desde el nodo recién expandido, se estima el resultado sin simular la partida completa. El *rollout* aplica **0-1 BFS** (Bloque 1) para comparar las distancias mínimas de ambos jugadores y determinar el ganador probable:

$$\text{resultado} = \begin{cases} \text{jugador_propio} & \text{si } \Delta d \geq \theta \\ \text{oponente} & \text{si } \Delta d \leq -\theta \\ \text{jugador con menor } d & \text{si } |\Delta d| < \theta \end{cases} \tag{14}$$

- $\Delta d = d_{\text{oponente}} - d_{\text{propio}}$: diferencia de distancias mínimas de conexión.
- θ: umbral adaptativo de decisión, hiperparámetro del Bloque 4:
  - $\theta = 2$ si $N > 13$: en tableros grandes, una diferencia de 2 pasos ya es estratégicamente decisiva.
  - $\theta = 3$ si $N \leq 13$: en tableros medianos se exige mayor margen para evitar falsos positivos.

Este *rollout* informado es sustancialmente más preciso que un *rollout* aleatorio y más rápido que simular jugadas hasta el estado terminal, permitiendo miles de iteraciones dentro del límite de tiempo.

Fase 4: Backpropagación

El resultado de la simulación se propaga hacia la raíz, actualizando las estadísticas de todos los nodos del camino recorrido:

$$w_v \mathrel{+}= \begin{cases} 1.0 & \text{si resultado} = \text{jugador_propio} \\ 0.5 & \text{si empate técnico} \\ 0.0 & \text{si resultado} = \text{oponente} \end{cases} \qquad n_v \mathrel{+}= 1 \tag{15}$$

El valor 0.5 para empate técnico (posición equilibrada según BFS) evita que el árbol descarte prematuramente ramas en posiciones cercanas al equilibrio.

Selección de la Jugada Final

Tras agotar el presupuesto de tiempo, el agente selecciona el hijo de la raíz con **mayor número de visitas**, no el de mayor *win-rate*:

$$a^* = \arg \max_{v \in \text{hijos(raíz)}} n_v \tag{16}$$

Este criterio es más robusto estadísticamente: un nodo muy visitado ha sido consistentemente seleccionado por UCB1, lo que implica que su *win-rate* ha sido estable bajo muchas estimaciones. Un nodo con pocas visitas pero alto win-rate puede tener ese valor por azar estadístico en muestras pequeñas.

Win-Rate Warmup: Reordenamiento Adaptativo

Una mejora adicional sobre el MCTS estándar conecta el Bloque 4 (metaheurística adaptativa) con el Bloque 1 (greedy): tras un número inicial de iteraciones de *warmup*, los movimientos aún no expandidos de la raíz se reordenan por su win-rate provisional acumulado durante esas primeras simulaciones.

Sea $\text{WARMUP} = \max(20, |\text{moves}|)$ el número de iteraciones de calentamiento. Para cada jugada m de la raíz se acumulan estadísticas w_m y n_m durante las simulaciones que la eligieron como primera jugada del jugador propio. Tras WARMUP iteraciones:

$$\text{untried_moves} \leftarrow \text{sort}\left(\text{untried_moves},\ \text{key} = -\frac{w_m}{n_m}\right) \tag{17}$$

- w_m: victorias acumuladas en simulaciones donde m fue la primera jugada propia.
- n_m: número de simulaciones donde m fue la primera jugada propia.
- La fracción w_m/n_m es el *win-rate* empírico de m según las simulaciones ya realizadas.

Este mecanismo es equivalente a **Hill-Climbing sobre el espacio de jugadas raíz**: en cada fase de reordenamiento se mueve hacia la configuración con mejor evaluación local (mayor win-rate), explotando la información estadística acumulada para guiar la expansión futura hacia las zonas más prometedoras del árbol. Las jugadas con mayor win-rate empírico se expandirán antes, acelerando la convergencia del árbol MCTS hacia la jugada óptima dentro del presupuesto de tiempo disponible.

Resumen del Pipeline MCTS

Componente	Origen	Función
UCB1 con $C = 0.8$	Bloque 4	Balance exploración/explotación
Rollout BFS	Bloque 1	Estimación rápida y precisa del resultado
Frontera BFS	Bloque 1	Candidatos en $O(k{\cdot}6)$ vs $O(N^2)$
Ordenamiento greedy raíz	Bloque 1	Inicialización informada de `untried_moves`
Win-rate warmup	Bloque 4	Reordenamiento adaptativo (Hill-Climbing)
Selección por visitas	Bloque 4	Robustez estadística en la jugada final

Table 5: Componentes del pipeline MCTS y su relación con los contenidos del curso.

Gestión del Tiempo y Correctitud

El reglamento del torneo establece un límite estricto de 5 segundos por jugada, con descalificación inmediata en caso de superarlo. La gestión del tiempo no es un detalle de implementación sino un requisito de correctitud: un agente que produce la jugada perfecta fuera de tiempo es equivalente a uno que no jugó. El diseño del sistema de tiempo atraviesa todas las capas del agente.

Arquitectura de Presupuestos de Tiempo

El agente asigna presupuestos de tiempo diferenciados según el rol del jugador, dejando un margen de seguridad sobre el límite de 5 segundos:

Parámetro	Jugador 1	Jugador 2
Presupuesto por turno	4.70 s	4.75 s
Margen de seguridad	0.30 s	0.25 s
`TIME_MARGIN` (corte de emergencia)	0.03 s	
`EARLY_STOP_MARGIN` (parada anticipada)	12% del presupuesto	

Table 6: Parámetros de gestión del tiempo del agente.

El Jugador 2 recibe un margen ligeramente mayor (4.75 s vs 4.70 s) porque en la práctica el segundo jugador tiende a enfrentarse a posiciones más complejas tácticamente, donde la búsqueda puede ser más costosa.

Niveles de Control de Tiempo

El sistema implementa tres niveles de control que actúan en distintos puntos del proceso de decisión:

Nivel 1: Parada Anticipada del IDAB

Al inicio de cada iteración del Iterative Deepening Alpha-Beta, se verifica si el tiempo restante supera el `EARLY_STOP_MARGIN`:

$$t_{\text{restante}} < t_{\text{límite}} \times 0.12 \implies \text{detener IDAB} \tag{18}$$

Si el tiempo restante es inferior al 12% del presupuesto total, no se inicia una nueva iteración aunque no se haya alcanzado $d_{\max}$. Esto garantiza que la iteración en curso siempre tenga tiempo suficiente para completarse, evitando devolver un resultado parcial de una profundidad mayor que podría ser peor que el de la iteración anterior completada.

Nivel 2: Corte de Emergencia en Alpha-Beta

Dentro de cada nodo del árbol Alpha-Beta, se verifica el tiempo antes de cada expansión:

$$t_{\text{restante}} < \texttt{TIME_MARGIN} = 0.03 \text{ s} \implies \text{evaluar heurísticamente y retornar} \tag{19}$$

Si el tiempo restante cae por debajo de 0.03 segundos en cualquier punto de la recursión, el nodo se evalúa con la función heurística en lugar de seguir expandiendo. Este mecanismo actúa como red de seguridad de último recurso: incluso si los niveles superiores no detuvieron la búsqueda a tiempo, el corte de emergencia garantiza que siempre se retorne una jugada dentro del límite.

Nivel 3: Criterio de Parada de MCTS

En el régimen MCTS, el criterio de parada es directamente temporal:

$$t_{\text{restante}} \leq 0.05 \text{ s} \implies \text{detener iteraciones MCTS} \tag{20}$$

El margen de 0.05 segundos para MCTS es mayor que el `TIME_MARGIN` de Alpha-Beta porque cada iteración MCTS implica un `board.clone()` y un BFS completo, operaciones con mayor variabilidad de tiempo que una evaluación heurística. El margen adicional absorbe esa variabilidad sin riesgo de descalificación.

Garantía de Jugada Válida

El agente garantiza en todo momento que `play()` devuelva una jugada válida mediante un sistema de respaldo en cascada:

1. **Jugada de apertura**: el centro del tablero se devuelve directamente si está disponible en los primeros turnos, sin invocar ningún algoritmo de búsqueda.
2. **Jugada forzada CSP**: si se detecta victoria inmediata o bloqueo urgente, se devuelve antes de invocar Minimax o MCTS.
3. **Mejor jugada del IDAB**: la jugada de la iteración completada más reciente, que siempre existe tras al menos una iteración de profundidad 1.
4. **Mejor hijo MCTS por visitas**: el hijo de la raíz con más visitas tras las simulaciones realizadas.
5. **Fallback**: si ningún mecanismo anterior produce una jugada (tablero casi lleno o estado inesperado), se devuelve la primera celda vacía encontrada en orden de recorrido.

Este sistema en cascada garantiza que `play()` nunca retorne `None` ni lance una excepción, cumpliendo el requisito de correctitud del proyecto independientemente del estado del tablero o del tiempo restante.

Correctitud e Idempotencia

Además de la gestión del tiempo, el agente satisface el requisito de **idempotencia** entre partidas: el objeto `SmartPlayer` puede disputar partidas sucesivas sin que el estado de una afecte a la siguiente. Al inicio de cada partida (detectado por `_move_count` ≤ 1), se reinician explícitamente todos los componentes de estado:

- **Tabla de transposición** (`_tt.clear()`): los valores evaluados de una partida no son válidos en otra, ya que corresponden a posiciones distintas del tablero.
- **Killer moves** (`_killers.clear()`): las jugadas que causaron cortes en partidas anteriores no son necesariamente útiles en la nueva partida.
- **Contadores incrementales**: `_my_count`, `_opp_count` y `_known_cells` se reinician a cero y vacío respectivamente, garantizando que el hash Zobrist incremental parta de una base correcta.

La **History Heuristic** (`_history`) es el único componente que *no* se reinicia entre partidas de forma deliberada: las celdas que estadísticamente han causado más cortes en partidas anteriores del mismo objeto siguen siendo relevantes, acumulando conocimiento táctico a lo largo de toda la sesión de juego.

Conclusiones

El desarrollo de este agente autónomo para HEX permitió aplicar de forma integrada y coherente los cinco bloques del curso (dados hasta el momento del desarrollo del proyecto), demostrando que los contenidos teóricos no son herramientas aisladas sino componentes complementarios de un sistema de decisión inteligente.

Sobre las Decisiones de Diseño

La decisión más determinante del proyecto fue reconocer que un único algoritmo no puede ser óptimo para todos los tamaños de tablero. La naturaleza de HEX impone un factor de ramificación del orden de N^2, lo que hace que Minimax sea preciso pero intratable para tableros grandes, y MCTS sea escalable pero estadísticamente ruidoso en tableros pequeños. La frontera en $N = 11$ no es arbitraria: es el punto donde el espacio de búsqueda supera lo que IDAB puede explorar de forma significativa dentro del límite temporal.

La integración del razonamiento CSP y la planificación STRIPS resultó ser la aportación más original del proyecto. El concepto de *landmark* — una celda que debe aparecer en todo plan ganador — no solo mejoró el ordenamiento de movimientos, sino que proporcionó al agente una comprensión estratégica del

tablero que va más allá de la evaluación heurística local. En particular, la detección de landmarks singleton mediante arc-consistency evitó pérdidas que habrían pasado desapercibidas incluso a profundidades elevadas de Minimax.

En definitiva, el agente desarrollado demuestra que es posible construir un jugador competitivo para HEX aplicando exclusivamente los contenidos del curso de Inteligencia Artificial, sin recurrir a técnicas externas. La clave no reside en ningún algoritmo individual, sino en la articulación coherente de búsqueda informada, razonamiento adversarial, satisfacción de restricciones, metaheurísticas y planificación como partes de un sistema unificado de decisión.